

APIs

**Como pedir comida para um servidor
sem invadir a cozinha**

Seminário de Engenharia de Software II

Enzo · Isaac · Joloano · Matheus

Cardápio do dia

- 00** História: das subrotinas às APIs modernas
- 01** API: interface, contrato e tipos
- 02** HTTP: pedido, resposta e APIs reais
- 03** Tecnologias e comunicação
- 04** REST × GraphQL
- 05** Especificar, proteger e evoluir

1949–1951: as peças reutilizáveis

1949 · EDSAC

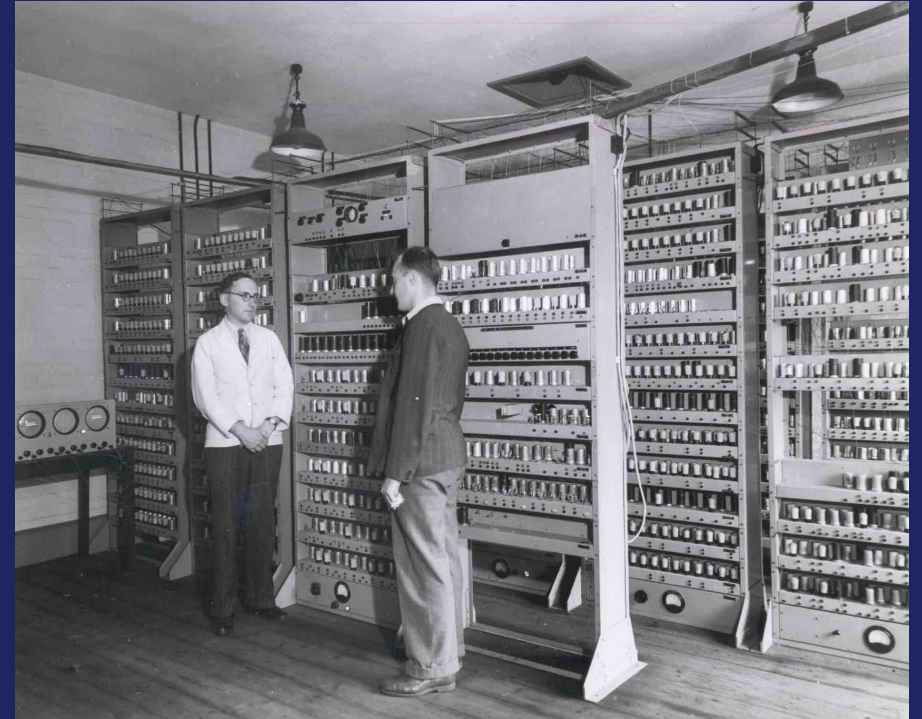
Em Cambridge, uma biblioteca de subrotinas permitia reaproveitar tarefas já programadas.

1951 · Um manual de programação

Wilkes, Wheeler e Gill explicaram como organizar e usar essas subrotinas.

A ideia de reutilizar funções antecede a web.

O “não vou fazer tudo do zero” tem história.



EDSAC · Cambridge

1984–1991: as chamadas chegam à rede

1984 • RPC

Birrell e Nelson detalham chamadas de procedimentos em outro computador. A rede entra no caminho da função.

1989 → 1991 • Web

Berners-Lee propõe a Web no CERN. Em 1991, ela começa a se espalhar para fora do laboratório.

O que muda?

Programas passam a acessar operações à distância. A Web oferece endereços e HTTP para a comunicação.

A função saiu para passear. Agora pode dar timeout.

1998–2000: XML, SOAP e REST

1998

XML-RPC

Chamadas remotas empacotadas em XML.

2000

SOAP 1.1

Mensagens com envelope e corpo;
cabeçalho opcional.

2000

REST

Fielding descreve um estilo arquitetural
com recursos e interface uniforme.

SOAP e REST coexistem. A internet não tem final de campeonato.

2011–2015: novas formas de conversar

Data	Marco	O que acrescenta
2011	WebSocket · RFC 6455	Canal persistente de mão dupla.
2012	GraphQL no Facebook	Início do desenvolvimento interno.
fev. 2015	gRPC aberto	Framework RPC baseado em HTTP/2.
jul. 2015	GraphQL aberto	Consulta de campos de um schema.
nov. 2015	OpenAPI Initiative	Descrição comum de APIs HTTP.

Mais ferramentas, mais escolhas. O “depende” ganhou currículo.

01

O que é uma API?

O cliente pede.
A cozinha se vira.



Você pede. Não invade a cozinha.



- 01 **Cliente faz o pedido**
- 02 **API expõe o contrato**
- 03 **Sistema executa**

API: Interface de Programação de Aplicações

A

Application: quem oferece ou consome uma função.

P

Programming: a conversa acontece por código.

I

Interface: o combinado entre os dois lados.

O ABC de uma API

A

Abstração

Você usa a função sem conhecer toda a implementação.

B

Baixo acoplamento

Cliente e servidor podem evoluir com menos dependência.

C

Contrato

Entradas, saídas e erros seguem regras conhecidas.

APIs conectam serviços e sistemas

Mapas

Um app consulta rotas e tempo de viagem.

Pagamentos

Uma loja solicita uma cobrança e recebe o resultado.

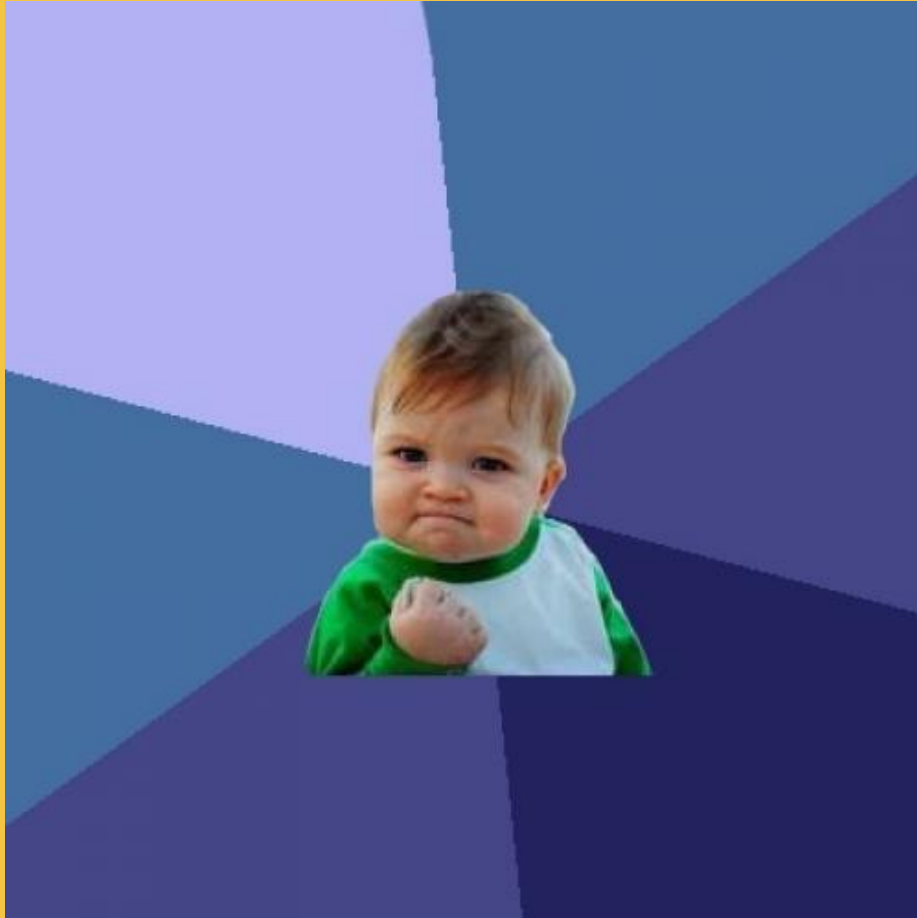
Identidade

Um serviço valida login e permissões.

Dentro da empresa

Estoque e vendas trocam dados entre sistemas.

Consumiu a API. Não invadiu o banco.



200 OK

```
GET /api/clientes/42
```

O banco do vizinho
continua em paz.

O restaurante, traduzido para software

Restaurante	Software
Cliente com fome	App / sistema consumidor
Cardápio	Contrato da API
Atendimento	Interface exposta
Cozinha	Implementação e regras
Pedido → prato	Requisição → resposta

A cozinha pode mudar. O combinado precisa continuar funcionando.

API também mora dentro de uma biblioteca.



`java.util.List...` isso é uma API?

Operações e dados

Quais ações existem, o que enviar e como será a resposta (schema).

Identidade e erros

Como autenticar, quais permissões exigir e qual formato usar nos erros.

Limites e evolução

Paginação, cota de chamadas e mudanças que preservam a compatibilidade.

APIs também vivem fora da web

API = interface de programação de aplicações.

Tipo	Exemplo de operação	Como o programa usa
Biblioteca	Java · pedidos.add(pedido)	Adiciona um item à lista dentro do próprio programa.
Sistema operacional	Windows · CreateFileW	Abre ou cria um arquivo e devolve um identificador.
Web	HTTP · POST /api/pedidos	Envia dados para uma operação de outro serviço pela rede.

A interface define operações e regras; o mecanismo da chamada muda.

Cada cozinha tem seu balcão.

Endpoint: onde o pedido chega

`https://api.loja.com/v1/pedidos/42`

Domínio + versão + recurso + identificador

GET /v1/pedidos/42

O método diz o que fazer; o caminho indica em qual recurso.

Google Cloud Endpoints

O caminho /v1/pedidos é um endpoint da API.

Cloud Endpoints é o produto que ajuda a publicar e gerenciar a API.

```
api.endpoints.meu-projeto.cloud.goog/v1/pedidos
```

Autenticação • cotas • métricas • OpenAPI

02

HTTP: o pedido por dentro



“me vê um risoto”

POST
/api/pedidos

Por que HTTP/1.1 foi adotado?

ANTES

Uma nova conexão TCP para cada pedido era comum no HTTP/1.0.

COM HTTP/1.1

Conexão persistente por padrão; o Host identifica o site no servidor.

Para APIs, isso trouxe uma base comum para métodos, cabeçalhos e respostas.

A requisição: “sem cebola” vai no body.

```
POST /api/pedidos?mesa=12
Authorization: Bearer <token>
Content-Type: application/json
Accept: application/json

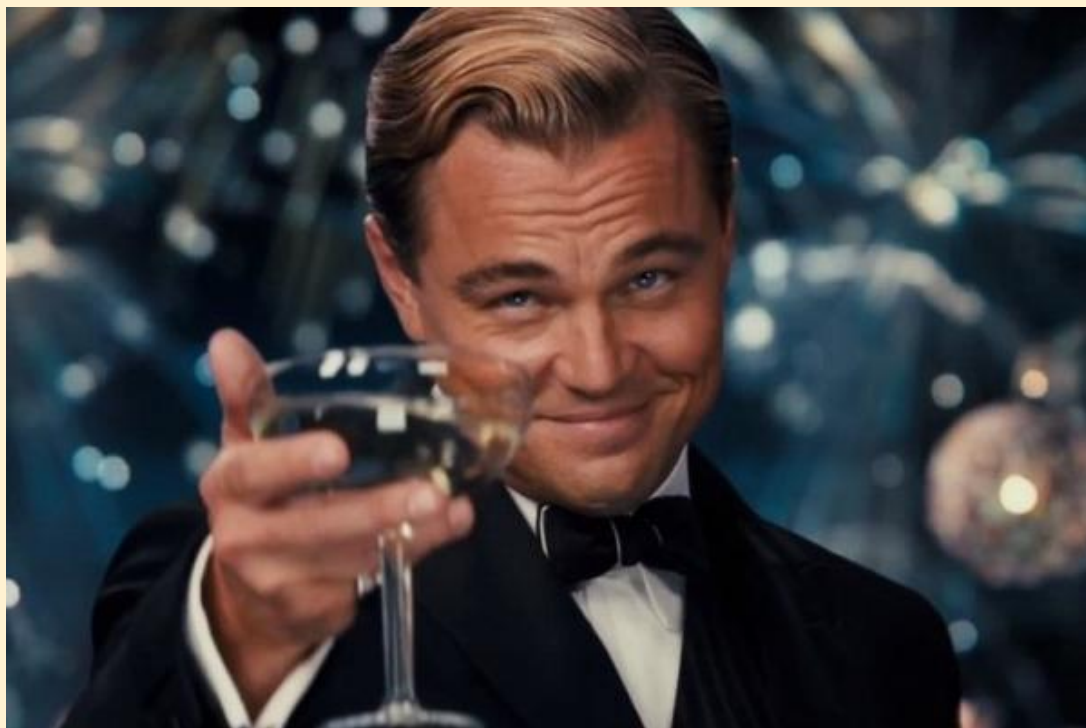
{ "prato": "risoto",
  "obs": "sem cebola" }
```

Path e query: recurso e parâmetros da chamada.
Authorization: credencial de quem faz o pedido.
Content-Type: formato enviado. Accept: formato aceito.
Body: dados que a operação precisa processar.



E a cebola veio no header?

201 Created: o pedido nasceu.



Um brinde ao JSON que chegou.

```
HTTP/1.1 201 Created  
Location: /api/pedidos/981  
Content-Type: application/json
```

```
{ "id": 981,  
  "status": "na_cozinha" }
```

201 confirma a criação.
Location aponta para o novo recurso.
Content-Type descreve o corpo recebido.
Em 204 No Content, a resposta vem sem corpo.

JSON e XML: dois formatos de retorno

JSON · application/json

```
{  
  "id": 42,  
  "status": "pronto"  
}
```

XML · application/xml

```
<pedido>  
  <id>42</id>  
  <status>pronto</status>  
</pedido>
```

O status HTTP diz como foi a requisição. O corpo carrega os dados.

Cinco verbos. Cinco intenções.

Método	Intenção	Idempotente?
GET	Consultar	Sim
POST	Processar / criar	Não garantido
PUT	Substituir / criar no endereço	Sim
PATCH	Alterar parte	Não garantido
DELETE	Remover	Sim

Idempotência: repetir tem o mesmo efeito pretendido.

POST tem mais de um emprego.



**POST só
cria coisas**



**POST processa
a operação
do recurso**

```
POST /pedidos  
POST /calcular-frete  
POST /checkout
```

**Repetiu o checkout?
O cartão sentiu.**

Status HTTP: os gatos explicam.



200
OK

2xx · Sucesso



404
Not Found

**4xx · Problema na
requisição**



500
Internal Server Error

5xx · Falha no servidor

Tier list totalmente imparcial™

Tier	Status	Tradução livre
S	200 / 201	Tudo certo
A	204	Feito. Sem textão.
B	404	Não encontrei
C	429	Calma, jovem
D	500	A cozinha parou



Uma conversa, duas direções.

CLIENTE

API / SERVIDOR

GET /usuarios/42 →

← 200 OK • { "nome": "Maria" }

GET /usuarios/999 →

← 404 Not Found

HTTPS: o 'S' usa TLS

HTTP + TLS = HTTPS

Sigilo: terceiros não leem o tráfego em trânsito.

Integridade: alterações no caminho são detectadas.

Certificado: ajuda o cliente a autenticar o servidor.

“SSL” é o nome antigo; o protocolo atual é TLS.

Token: mostre que você pode entrar

Authorization:
Bearer <token>

O servidor confere validade, escopo e permissão para o recurso.

HTTPS protege o token enquanto ele viaja.



F12: descobrindo a conversa escondida.

- 01 **Abra o DevTools**
- 02 **Rede / Network**
- 03 **Filtre Fetch/XHR**
- 04 **Veja pedido e resposta**



“O site chama tudo isso?”

GitHub: lendo a resposta de uma API

```
GET https://api.github.com/repos/octocat/Hello-World
```

```
200 OK · application/json
```

```
{
  "id": 1296269,
  "name": "Hello-World",
  "full_name": "octocat/Hello-World",
  "private": false,
  "html_url": "https://github.com/octocat/Hello-World",
  "description": "My first repository on GitHub!",
  "default_branch": "master"
}
```

A imagem mostra os dados devolvidos pela API.

Campos selecionados de uma consulta real · 25/09/2026

Pedido

GET consulta o repositório octocat/Hello-World.

Resposta

200 indica sucesso.
O corpo contém JSON.

Uso no app

Ler name e description
para preencher uma tela.

ViaCEP: um CEP entra, um endereço sai

```
{
  "cep": "01001-000",
  "logradouro": "Praça da Sé",
  "complemento": "lado ímpar",
  "unidade": "",
  "bairro": "Sé",
  "localidade": "São Paulo",
  "uf": "SP",
  "estado": "São Paulo",
  "regiao": "Sudeste",
  "ibge": "3550308",
  "gia": "1004",
  "ddd": "11",
  "siafi": "7107"
}
```

1 • Entrada

GET /ws/01001000/json/
O caminho informa o CEP
e o formato esperado.

2 • Saída

O JSON traz logradouro, bairro,
cidade e UF para o programa ler.

3 • Uso prático

Um formulário pode preencher
o endereço depois de ler o CEP.

03

Mapa das tecnologias

Nomes parecidos.
Papéis diferentes.



Seis nomes. Categorias diferentes.

Nome	O que é	Ideia central
REST	Estilo arquitetural	Recursos + restrições
SOAP	Protocolo de mensagens	Envelope XML
gRPC	Framework de RPC	Chamar funções remotas
GraphQL	Linguagem + execução	Consultar um schema
WebSocket	Protocolo	Canal bidirecional
Webhook	Padrão de eventos	Provedor chama sua URL

SOAP: mensagens com estrutura XML

SOAP organiza a troca de mensagens entre sistemas.

Envelope

A embalagem da mensagem.

Header

Metadados opcionais, como segurança.

Body

A operação ou resposta enviada.

Exemplo: integração entre sistemas corporativos com contrato bem definido.

SOAP chegou de terno para a integração



O pedido é formal:

```
<env:Envelope>  
  <env:Header>...</env:Header>  
  <env:Body>  
    <consultarPedido/>  
  </env:Body>  
</env:Envelope>
```

A resposta também segue um envelope XML.

Um app. Várias conversas.



REST

cardápio e pedidos

GraphQL

dados para uma tela

SOAP

integração fiscal

gRPC

serviços internos

WebSocket

mapa ao vivo

Webhook

pagamento aprovado

“Já ficou pronto?”



POLLING

Perguntas em intervalos.

WEBSOCKET

Canal aberto, nos dois sentidos.

WEBHOOK

O evento chama o seu servidor.

O dilema do dev com pressa.



App: “já tá pronto?”

Servidor: “não.”

App: “e agora?”

429 Too Many Requests

WebSocket: a linha fica aberta.



- 01 Chat, jogos, mapa ao vivo
- 02 Autenticar e reconectar
- 03 wss:// para usar TLS

Webhook: “fica tranquilo, eu te aviso”.



Esperar ≠ perguntar sem parar

- 01 **Cadastre a URL**
- 02 **O evento acontece**
- 03 **Receba o POST**

Assinatura ✓ Duplicatas ✓
2xx rápido → processe na fila

WebSocket × Webhook

	WebSocket	Webhook
Conexão	Persistente	Requisições por evento
Direção	Bidirecional	Provedor → receptor
Uso comum	App ↔ servidor	Servidor ↔ servidor
Exemplo	Chat / mapa ao vivo	Pagamento aprovado
Falhas	Reconectar o canal	Retries conforme provedor

Rádio ligado × telefonema quando o pedido fica pronto.

04

REST × GraphQL

“Qual é melhor?”
O sênior: depende.



HTTP + JSON ≠ REST

Mude minha ideia.



Cliente–servidor

Interface e execução têm papéis separados.

Stateless

Cada pedido traz o contexto necessário.

Cache

A resposta indica se pode ser reutilizada.

Interface uniforme

Recursos, mensagens autodescritivas e links.

Camadas

Proxies e gateways podem intermediar.

Código sob demanda (opcional)

O servidor pode enviar código executável.

Um recurso. Um endereço. Um contrato.

```
GET /usuarios/42
Accept: application/json
Authorization: Bearer <token>

200 OK
Cache-Control: private, max-age=60

{ "id": 42, "nome": "Maria" }
```



**Cada pedido
se explica sozinho.**

GraphQL: peça os campos que precisa.



Eu só precisava do nome...

```
query {  
  usuario(id: 42) {  
    nome  
    pedidos { total }  
  }  
}
```

A escolha tem limites

Campos definidos pelo schema.
Permissão validada no servidor.
Profundidade e custo controlados.

Pode reunir dados em uma consulta.

REST × GraphQL: escolha pelo problema.

Critério	REST	GraphQL
Resposta	Definida pelo endpoint	Campos escolhidos no schema
Cache	HTTP bem estabelecido	Depende da estratégia
Custo	Por operação / recurso	Custo de consultas e resolvers
Evolução	Compatibilidade e versões	Adicionar / depreciar campos
Uso frequente	Integrações e recursos	Dados relacionados em telas

Podem coexistir. “Depende” continua sendo uma resposta válida.

05

Engenharia do contrato

“Não basta funcionar
na sua máquina.”



Sem documentação? Não passa.

OpenAPI: o contrato ganha ferramentas.

OpenAPI descreve o contrato HTTP. Abaixo: Swagger UI da API de exemplo Petstore.



Documentação

Veja endpoints, parâmetros e respostas no Swagger UI.

SDKs

Gere clientes a partir do contrato publicado.

Mocks

Simule respostas para o front começar antes.

Testes de contrato

Compare implementação e especificação no CI.

Contrato: o combinado de um endpoint

```
POST /api/pedidos?mesa=12
```

Parte	Regra deste exemplo
Entrada	mesa: inteiro positivo; JSON: PedidoEntrada.
Acesso	Authorization: Bearer <token>.
Sucesso	201 Created + Location + JSON com id e status.
Erros previstos	400: JSON inválido · 401: credencial inválida 403: sem permissão · 422: dados fora das regras.

O contrato também define o formato dos erros e os limites de uso.

Schema: o JSON tem regras

Campo	Tipo e regra
prato	Texto obrigatório · 1 a 60 caracteres.
quantidade	Inteiro de 1 a 10 · opcional; o serviço assume 1.
obs	Texto opcional · até 120 caracteres.

VÁLIDO

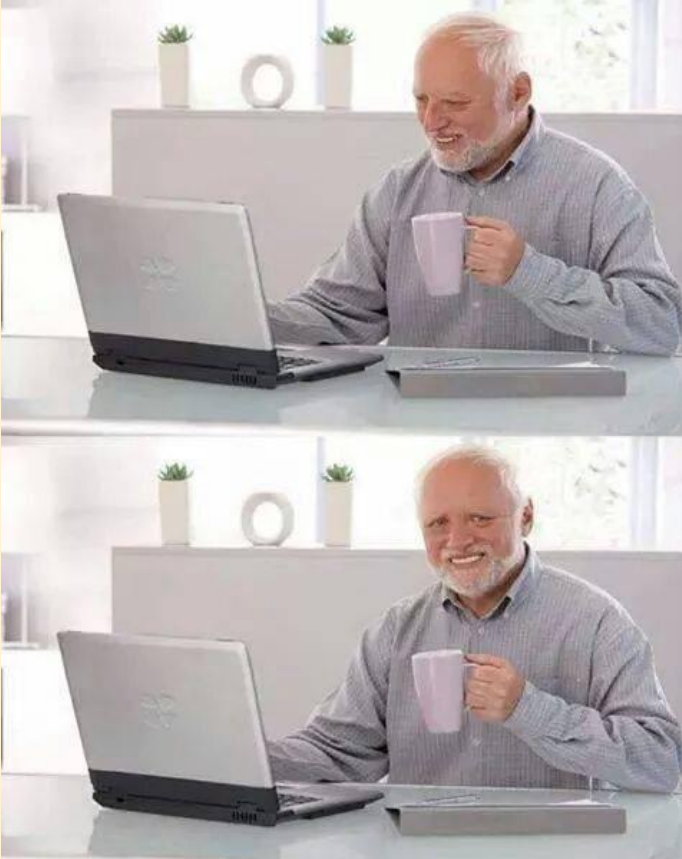
```
{ "prato": "risoto",  
  "quantidade": 2 }
```

INVÁLIDO

```
{ "prato": "",  
  "quantidade": "duas" }
```

O servidor valida. JSON válido ainda pode pedir um prato esgotado.

A documentação × a produção



PROMESSA

Pedido inexistente → 404

REALIDADE

```
200 OK
{ "sucesso": false,
  "erro": "deu ruim" }
```

O plano infalível para quebrar clientes.



01 **Preserve
compatibilidade**

02 **Avise e dê prazo**

03 **Versione quando
preciso**

Sexta, 18h03: “só renomeei um campo”.



nomeUsuario



userName

18h04

TypeError

Pedido seguro. Cobrança só uma vez.



HTTPS e autorização

Proteja os dados em trânsito e confira se o usuário pode acessar cada objeto.

Entrada e limites

Valide o body e limite chamadas excessivas (429).

Timeout e retry

Defina tempo máximo para cada chamada.
Repita operações idempotentes com espera crescente.

Idempotência

Use Idempotency-Key quando a API oferecer esse recurso, para evitar cobranças duplicadas.

Quando o ID muda... e a API deixa.



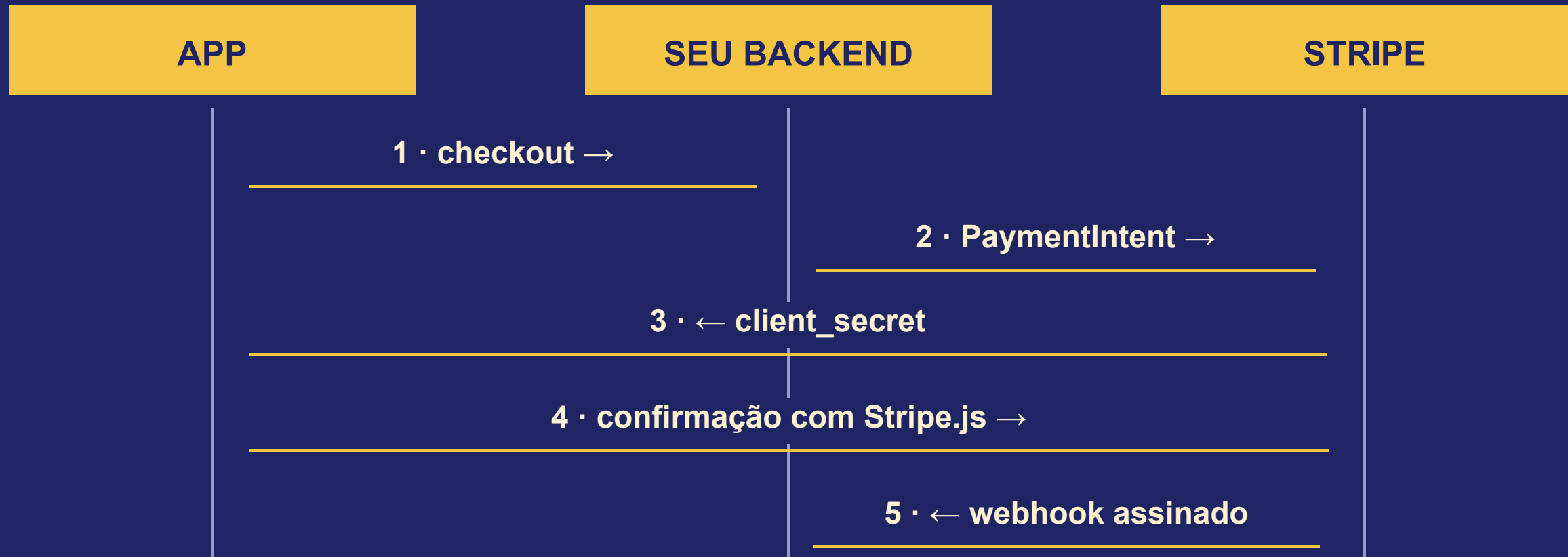
```
GET /usuarios/42
```

```
GET /usuarios/43
```

**Login não é permissão
para ver qualquer conta.**

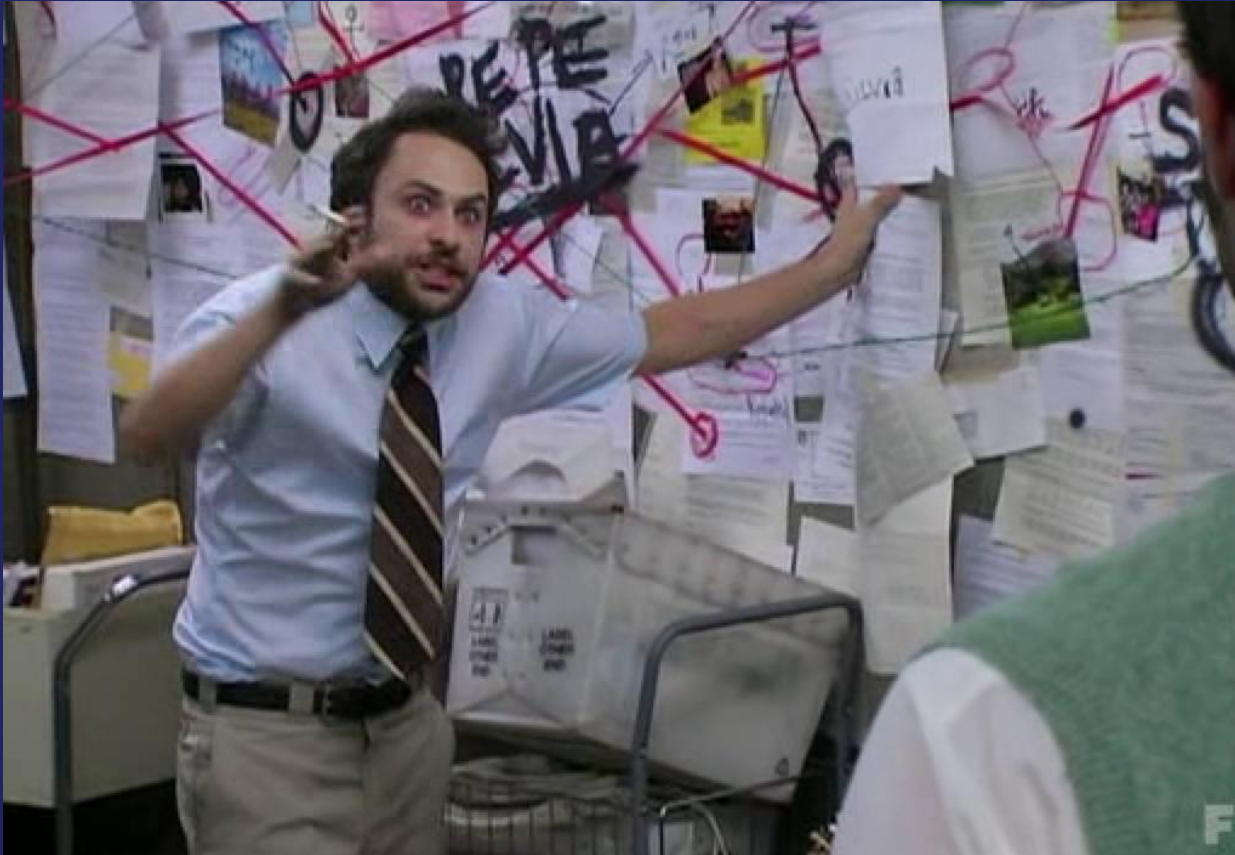
O servidor precisa verificar o acesso a cada objeto.

Um fluxo real: pagamento com Stripe.



Backend cria • app confirma • webhook informa o resultado final.

Quem chama quem nessa arquitetura?



- 01 **Monólito também expõe API**
- 02 **Gateway: entrada e políticas**
- 03 **BFF: API adaptada à tela**
- 04 **Eventos: avisos entre sistemas**

“Vamos dividir em microserviços!”



**47 serviços
12 filas
3 gateways**

**Quem chama quem?
Boa pergunta.**

Se lembrar de três coisas...

01 API é contrato.

02 Tecnologia depende da conversa.

03 Bom contrato aguenta mudança e falha.

Agora vocês são a API.



REQUISIÇÕES. REQUISIÇÕES POR TODA PARTE.

- 01 Definam o cardápio
- 02 Façam uma requisição
- 03 Devolvam um status
- 04 Simulem um erro

Perguntas?

200 OK: a gente responde.
202 Accepted: a gente pesquisa.

Obrigado!

Enzo · Isaac · Joloano · Matheus

Para fechar...

BINGO DAS APIs

Ouçá a definição, descubra o termo e marque na cartela.

A ideia é revisar os conceitos principais de APIs sem computador, usando as definições trabalhadas na apresentação.

B I N G O

APIs
